ls
grep
true
grep
complex_command
nl
python3
python3
nl
grep
nl
nl
nl
nl
sed
sed
nl
python3
python3
python3
echo
git
git
nl
python3
python3
nl
grep
grep
1
1
2
2
3
4
5
6
7
8
8
9
10
11
12
13
14
14
15
16
17
18
19
19
19
20
21
22
23
24
24
24
localization
patch
validation
general

```
.../ask_generated.py')s = p.read_text()targets = ['q.even: set([', 'q.odd: set([', 'q.zero: set([']for t in targets:    idx = s.find(t)    if idx == -1:        continue    start = idx + len(t)    end = s.find('])', start)    if end == -1:        continue    segment = s[start:end]    if 'q.finite' not in segment:        # insert q.finite right after the opening bracket, keeping formatting        s = s[:start] + 'q.finite, ' + s[start:]p.write_text(s)print('modified', p)
```

```
.../from sympy import symbolm = symbol('m', even=true)print("m.is_even:", m.is_even)print("m.is_finite:", m.is_finite)
```

```
.../assumptions.py")# show the relevant section to verifyimport sysprint("--- snippet ---")for i, line in enumerate(s_new.splitlines()[170:196], start=171):    print(f"{i:4d}  {line}")# now test the behaviorfrom sympy import symbolm = symbol('m', even=true)print("m.is_even:", m.is_even)print("m.is_finite:", m.is_finite)
```

```
.../n")    print("repaired assumptions.py")else:    print("no change needed")# show the relevant snippetprint("--- snippet ---")for idx, line in enumerate(open(p).read().splitlines()[180:196], start=181):    print(f"{idx:4d}  {line}")# test behaviorfrom sympy import symbolm = symbol('m', even=true)print("m.is_even:", m.is_even)print("m.is_finite:", m.is_finite)
```

```
.../test failed:", e)
```

```
.../n        ~q.even | q.finite,')    p.write_text(s)    print('inserted cnf clause for even -> finite')else:    print('clause already present')# now test the effectfrom sympy import symbolm = symbol('m', even=true)print("m.is_even:", m.is_even)print("m.is_finite:", m.is_finite)
```

```
.../from sympy.assumptions.ask_generated import get_known_facts_dict, get_known_facts_cnffrom sympy import symbolfrom sympy.assumptions.ask import q, askd = get_known_facts_dict()print("compressed facts for q.even:", d.get(q.even))print("does cnf include '~q.even | q.finite'?", any(str(c).find("~q.even | q.finite")!=-1 for c in get_known_facts_cnf().args))m = symbol('m', even=true)print("symbol m assumptions dict:", m.assumptions0)print("ask(q.finite, m):", ask(q.finite, m))print("m.is_finite:", m.is_finite)
```